

LLDD BE - API Report and Master Data

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจสอบ และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. Overview

รายการ	รายละเอียด
Track	BE
Estimate	39 ชั่วโมง = implementation 30 + unit test 9 (30%)
Owner	Peerakorn <Pete> Sakunkaewphithak
Target repository	SBP/srm-sps-spsap-store-backend (NestJS + TypeORM · schema sps_store) + SBP/srm-sps-spsap-sbp-bff (forward ผ่าน client service · ไม่มี DB) สำหรับเส้นที่ FE เรียก
Objective	ออกแบบ APIs สำหรับรายงานตรวจสอบประกันรายได้ และ Master Data ที่ SBPGI ดูแลเอง (ปัจจัยภายนอก + รายชื่อคู่แข่ง)

Common contract reference: ทุกหัวข้อ API/FE ต้องยึด LLDD-BE-API-Common-Contracts.pdf และ LLDD-FE-Integration-Contracts.pdf สำหรับ error/auth/format/pagination/action/RBAC ก่อนลงรายละเอียดเฉพาะหน้าหรือเฉพาะ endpoint

2. Screen / Functional Scope

- Report query service
- Excel export (14 columns, SDD slide 60)
- Operator/factor CRUD
- Report filters

3. Screenshot Reference

ไม่มีภาพหน้าจอสำหรับหัวข้อนี้ — เป็นเอกสารฝั่ง Backend/Batch ที่ไม่มี UI (ภาพหน้าจอทั้งหมดอยู่ในเอกสารชุด FE)

4. Implementation Flow Diagram (Reference)

LLDD BE - API Report and Master Data



รูปที่ 1: Implementation flow reference: LLDD BE - API Report and Master Data

5. Field, Format, and Validation

Field / UI	Format	Validation	Behavior
year	ค.ศ. YYYY	required for report	return 400 if missing · BE ผ่าน toAD() เพื่อ client ส่ง พ.ศ.
status	statusCode string	required	6 สถานะเอกสาร; verbatim จาก sps_store.workflow_status ของ @srm/glb-workflow
result	APPROVE REJECT CANCELLED PENDING	optional for report (บังคับเฉพาะ status)	maps to consideration_logs.result_category ล่าสุด · CANCELLED = ยกเลิกโดยระบบ (เพิ่ม 2026-08-10)
region	array/string	optional	13 region codes; multi-select
storeType	array ของ BranchTypeFGIName	optional	7 ค่า A B C D E PTT บริษัท (ยืนยันจาก master BranchTypeProfile ของ CPA_FRN_FGI 2026-08-10) · multi-select · ห้าม hardcode ให้โหลดจาก GET /common/common-code ของระบบ SBP เดิม
impactedStoreCode	string 5 digits	optional	คง leading zero
newStoreCode	string 5 digits	optional	คง leading zero
reason	text	required mutation	audit reason
page/size	integer	page>=1 size<=100	pagination

5.9 Input / Progress / Output Contract

Stage	Contract for implementation
Input	GET /api/v1/reports/status-summary; GET /api/v1/reports/status-summary/export; GET /api/v1/factors
Progress	Validate filter; Build query; Apply pagination/export mode; Return rows or CSV
Output	external_factors; competitors

5.90 Endpoint Implementation Contract

Endpoint	Use-case owner	Service/repository behavior	Definition of done
GET /api/v1/reports/status-summary	รายงานตรวจสอบประกันรายได้	Validate filter	missing year/status/result fails
GET /api/v1/reports/status-summary/export	Export Excel	Build query	export uses same filters as preview
GET /api/v1/factors	อ่านปัจจัยภายนอก	Apply pagination/export mode	master edit requires reason
POST /api/v1/factors	สร้างปัจจัยภายนอก	Return rows or CSV	config locked value cannot edit
PUT /api/v1/factors/{code}	แก้ปัจจัยภายนอก	For mutations validate reason and write audit	missing year/status/result fails
GET /api/v1/competitors	master แบนด์คู่แข่ง 11 รายการ (รหัส 01-11) — เป็นแหล่งของ dropdown รานคู่แข่งในหน้าเอกสารด้วย	Validate filter	export uses same filters as preview
POST /api/v1/competitors	เพิ่มแบนด์คู่แข่ง — code/nameTh/nameEn บังคับ · รหัสซ้ำตอบ 409	Build query	master edit requires reason

Endpoint	Use-case owner	Service/repository behavior	Definition of done
PUT /api/v1/competitors/{code}	แก้ไข/สถานะ — ห้ามแก้ code เพราะถูกอ้างอิงจาก document_competitors	Apply pagination/export mode	config locked value cannot edit
DELETE /api/v1/competitors/{code}	ลบแบรนด์คู่แข่ง — ถูกอ้างอิงในเอกสารแล้วตอบ 409	Return rows or CSV	missing year/status/result fails
DELETE /api/v1/factors/{code}	ลบปัจจัยภายนอกที่ไม่ถูกใช้งาน	For mutations validate reason and write audit	export uses same filters as preview

5.91 Backend Execution Sequence

Step	Behavior specific to this LLDD	Failure/test evidence
1	Validate filter	report missing year
2	Build query	report export
3	Apply pagination/export mode	factor duplicate
4	Return rows or CSV	operator audit
5	For mutations validate reason and write audit	config locked

6. Button / User Action Mapping

Action	Trigger	API / Service	Expected Result
Report preview	GET	report.service.search	paginated rows
Report export	GET	report.service.exportCsv	csv stream
Master mutation	POST/PUT/DELETE	master.service.save	อัปเดต row ของ master

7. API Contract

GET /api/v1/reports/status-summary

รายงานตรวจสอบประกันรายได้

Query Params
<pre>{ "year": 2026, "status": "06", "result": "APPROVE", "region": ["RSU"], "storeType": ["A"], "impactedStoreCode": "00788", "newStoreCode": "00990", "page": 1, "size": 20 }</pre>

Request Field Schema

Field	Type	Required	Constraint / Meaning
year	integer	Yes	UTF-8; use value domain described by endpoint purpose
status	string	Yes	UTF-8; use value domain described by endpoint purpose

Field	Type	Required	Constraint / Meaning
result	string	No	UTF-8; use value domain described by endpoint purpose
region	array<string>	No	JSON array; element type shown in Type column
storeType	array<string>	No	JSON array; element type shown in Type column
impactedStoreCode	string	No	exactly 5 digits; preserve leading zero
newStoreCode	string	No	exactly 5 digits; preserve leading zero
page	integer	No	>= 1; default 1
size	integer	No	1..100; default 20

Response

```
{
  "page": 1,
  "size": 20,
  "total": 0,
  "items": []
}
```

Response Field Schema

Field	Type	Required	Constraint / Meaning
page	integer	Yes	>= 1; default 1
size	integer	Yes	1..100; default 20
total	integer	Yes	UTF-8; use value domain described by endpoint purpose
items	array<object>	Yes	JSON array; element type shown in Type column

GET /api/v1/reports/status-summary/export

Export Excel

Query Params

```
{
  "year": 2026,
  "status": "06",
  "result": "APPROVE",
  "region": [
    "RSU"
  ],
  "storeType": [
    "A"
  ],
  "impactedStoreCode": "00788",
  "newStoreCode": "00990"
}
```

Request Field Schema

Field	Type	Required	Constraint / Meaning
year	integer	Yes	UTF-8; use value domain described by endpoint purpose
status	string	Yes	UTF-8; use value domain described by endpoint purpose

Field	Type	Required	Constraint / Meaning
result	string	No	UTF-8; use value domain described by endpoint purpose
region	array<string>	No	JSON array; element type shown in Type column
storeType	array<string>	No	JSON array; element type shown in Type column
impactedStoreCode	string	No	exactly 5 digits; preserve leading zero
newStoreCode	string	No	exactly 5 digits; preserve leading zero

Response

```
{
  "fileName": "insurance-verification-2026.xlsx"
}
```

Response Field Schema

Field	Type	Required	Constraint / Meaning
fileName	string	Yes	UTF-8; use value domain described by endpoint purpose

GET /api/v1/factors

อ่านปัจจัยภายนอก

Query Params

```
{
  "q": "ก่อสร้าง",
  "active": true,
  "page": 1,
  "size": 20
}
```

Request Field Schema

Field	Type	Required	Constraint / Meaning
q	string	No	UTF-8; use value domain described by endpoint purpose
active	boolean	No	UTF-8; use value domain described by endpoint purpose
page	integer	No	>= 1; default 1
size	integer	No	1..100; default 20

Response

```
{
  "page": 1,
  "size": 20,
  "total": 1,
  "items": [
    {
      "factorCode": "ROAD",
      "factorName": "ก่อสร้างถนน",
      "description": "ปิดช่องทางจราจร",
      "active": true
    }
  ]
}
```

Response Field Schema

Field	Type	Required	Constraint / Meaning
page	integer	Yes	>= 1; default 1
size	integer	Yes	1..100; default 20
total	integer	Yes	UTF-8; use value domain described by endpoint purpose
items	array<object>	Yes	JSON array; element type shown in Type column
items[].factorCode	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].factorName	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].description	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].active	boolean	Yes	UTF-8; use value domain described by endpoint purpose

POST /api/v1/factors

สร้างปัจจัยภายนอก

Request

```
{
  "factorCode": "ROAD",
  "factorName": "ก่อสร้างถนน",
  "description": "ปิดช่องทางจราจร",
  "active": true,
  "reason": "เพิ่มปัจจัยใหม่"
}
```

Request Field Schema

Field	Type	Required	Constraint / Meaning
factorCode	string	Yes	UTF-8; use value domain described by endpoint purpose
factorName	string	Yes	UTF-8; use value domain described by endpoint purpose
description	string	Yes	UTF-8; use value domain described by endpoint purpose
active	boolean	Yes	UTF-8; use value domain described by endpoint purpose

Field	Type	Required	Constraint / Meaning
reason	string	Yes	trimmed UTF-8 Thai text; required by operation/business rule

Response

```
{
  "factorCode": "ROAD",
  "factorName": "ก่อสร้างถนน",
  "active": true
}
```

Response Field Schema

Field	Type	Required	Constraint / Meaning
factorCode	string	Yes	UTF-8; use value domain described by endpoint purpose
factorName	string	Yes	UTF-8; use value domain described by endpoint purpose
active	boolean	Yes	UTF-8; use value domain described by endpoint purpose

PUT /api/v1/factors/{code}

แก้ไขปัจจัยภายนอก

Request

```
{
  "factorName": "ก่อสร้างและปิดถนน",
  "description": "มีดช่องทางจราจรบางส่วน",
  "active": true,
  "reason": "ปรับค่าอธิบาย"
}
```

Request Field Schema

Field	Type	Required	Constraint / Meaning
factorName	string	Yes	UTF-8; use value domain described by endpoint purpose
description	string	Yes	UTF-8; use value domain described by endpoint purpose
active	boolean	Yes	UTF-8; use value domain described by endpoint purpose
reason	string	Yes	trimmed UTF-8 Thai text; required by operation/business rule

Response

```
{
  "factorCode": "ROAD",
  "factorName": "ก่อสร้างและปิดถนน",
  "active": true
}
```


Response Field Schema

Field	Type	Required	Constraint / Meaning
factorCode	string	Yes	UTF-8; use value domain described by endpoint purpose
factorName	string	Yes	UTF-8; use value domain described by endpoint purpose
active	boolean	Yes	UTF-8; use value domain described by endpoint purpose

GET /api/v1/competitors

master แบรนดิ์คู่แข่ง 11 รายการ (รหัส 01-11) — เป็นแหล่งของ dropdown ร้านคู่แข่งในหน้าเอกสารด้วย

Query Params
<pre>{ "q": "108" }</pre>

Request Field Schema

Field	Type	Required	Constraint / Meaning
q	string	No	UTF-8; use value domain described by endpoint purpose

Response
<pre>{ "items": [{ "code": "01", "nameTh": "108 Shop", "nameEn": "108 Shop", "remark": "", "isActive": true }] }</pre>

Response Field Schema

Field	Type	Required	Constraint / Meaning
items	array<object>	Yes	JSON array; element type shown in Type column
items[].code	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].nameTh	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].nameEn	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].remark	string	Yes	UTF-8; use value domain described by endpoint purpose
items[].isActive	boolean	Yes	UTF-8; use value domain described by endpoint purpose

POST /api/v1/competitors

เพิ่มแบรนด์คู่แข่ง — code/nameTh/nameEn บังคับ · รหัสซ้ำตอบ 409

Request

```
{
  "code": "12",
  "nameTh": "ร้านค้าแข่งรายใหม่",
  "nameEn": "New Competitor",
  "remark": ""
}
```

Request Field Schema

Field	Type	Required	Constraint / Meaning
code	string	Yes	UTF-8; use value domain described by endpoint purpose
nameTh	string	Yes	UTF-8; use value domain described by endpoint purpose
nameEn	string	Yes	UTF-8; use value domain described by endpoint purpose
remark	string	Yes	UTF-8; use value domain described by endpoint purpose

Response

```
{
  "code": "12",
  "message": "saved"
}
```

Response Field Schema

Field	Type	Required	Constraint / Meaning
code	string	Yes	UTF-8; use value domain described by endpoint purpose
message	string	Yes	UTF-8; use value domain described by endpoint purpose

PUT /api/v1/competitors/{code}

แก้ไข/สถานะ — ห้ามแก้ code เพราะถูกอ้างอิงจาก document_competitors

Request

```
{
  "nameTh": "ลอร์สัน 108",
  "nameEn": "Lawson 108",
  "remark": "",
  "isActive": true
}
```

Request Field Schema

Field	Type	Required	Constraint / Meaning
nameTh	string	Yes	UTF-8; use value domain described by endpoint purpose
nameEn	string	Yes	UTF-8; use value domain described by endpoint purpose
remark	string	Yes	UTF-8; use value domain described by endpoint purpose

Field	Type	Required	Constraint / Meaning
isActive	boolean	Yes	UTF-8; use value domain described by endpoint purpose

Response

```
{
  "message": "saved"
}
```

Response Field Schema

Field	Type	Required	Constraint / Meaning
message	string	Yes	UTF-8; use value domain described by endpoint purpose

DELETE /api/v1/competitors/{code}

ลบแบรนด์คู่แข่ง — ถูกอ้างในเอกสารแล้วตอบ 409

Request

```
{}
```

Request Field Schema

Field	Type	Required	Constraint / Meaning
-	none	No	No fields

Response

```
{
  "message": "deleted"
}
```

Response Field Schema

Field	Type	Required	Constraint / Meaning
message	string	Yes	UTF-8; use value domain described by endpoint purpose

DELETE /api/v1/factors/{code}

ลบปัจจัยภายนอกที่ไม่ถูกใช้งาน

Request

```
{
  "reason": "ยกเลิกค่าทดสอบ"
}
```

Request Field Schema

Field	Type	Required	Constraint / Meaning
reason	string	Yes	trimmed UTF-8 Thai text; required by operation/business rule

Response
<pre>{ "factorCode": "ROAD", "deleted": true }</pre>

Response Field Schema

Field	Type	Required	Constraint / Meaning
factorCode	string	Yes	UTF-8; use value domain described by endpoint purpose
deleted	boolean	Yes	UTF-8; use value domain described by endpoint purpose

8. Reference DB Mapping (No Database Page Work)

ส่วนนี้เป็นข้อมูลอ้างอิงสำหรับการ implement API/Job เท่านั้น ไม่ใช้งานสร้างหน้า Database, ไม่ใช้งานออกแบบ DB page และไม่ถูกนับเป็น deliverable แยกของ FE/BE

Table / Object	R/W	Usage
compensation_documents	R	แหล่งข้อมูลรายงานและ filter status/year
compensation_histories	R	ยอดเงินชดเชยและงวด statement
consideration_logs	R	ผลพิจารณาล่าสุด APPROVE/REJECT
auth-backend group + scope (business_user_group) / prepared approver ของ @srm/glb-workflow	R	ผู้ปฏิบัติงาน — ตาราง operator_assignments ถูกตัด 2026-08-05
external_factors	R/W	master บัญชีภายนอก
competitors	R/W	master แปรนต์คู่แข่ง 11 รายการ (code 01-11 · name_th · name_en · remark) — feed dropdown รานคู่แข่งของหน้าเอกสาร
document_competitors	R	ตรวจว่าแปรนต์ถูกอ้างในเอกสารก่อนลบ (409)
mas_param (SBP)	R	ค่ากำหนดกลางของระบบ SBP เดิม — อ่านอย่างเดียว (หน้า Global Config ของ SBPGI ถูกลบ 2026-08-06 · ระบบเดิมเป็นผู้แก้)

9. Skeleton Code (store-backend + BFF)

โครงโค้ดตั้งต้นของเอกสารฉบับนี้ ยึด convention จริงของ srm-sps-spsap-store-backend (NestJS 11 + TypeORM, schema sps_store, custom provider DATA_SOURCE ที่ route SELECT ไป slave pool) และ srm-sps-spsap-sbp-bff (ไม่มี DB, forward ผ่าน client service). ทุกจุดที่ต้องเติมกำกับด้วย // TODO: และ response ทุกเส้นถูกห่อเป็น {success, data} โดย ResponseInterceptor อยู่แล้ว จึงห้าม service ห่อซ้ำ

9.1 ไฟล์ที่ต้องสร้าง

Path	หน้าที่
store-backend · src/modules/sbpgi-report-and-master-data/sbpgi-report-and-master-data.controller.ts	route ทั้งหมดของเอกสารนี้ (10 เส้น) + @UseGuards(HttpAuthGuard) + @UserId()
store-backend · src/modules/sbpgi-report-and-master-data/sbpgi-report-and-master-data.service.ts	business logic — inject 'DATA_SOURCE' แล้วยิง raw SQL, mutation ใช้ QueryRunner transaction
store-backend · src/modules/sbpgi-report-and-master-data/sbpgi-report-and-master-data.sql.ts	เก็บ SQL ต่อ endpoint (คัดจากหัวข้อ 10) แยกออกจาก service ให้ทดสอบ/รีวิวง่าย
store-backend · src/modules/sbpgi-report-and-master-data/dto/sbpgi-report-and-master-data.dto.ts	DTO + class-validator ตาม validation ในหัวข้อฟิลด์ของเอกสารนี้

Path	หน้าที่
store-backend · src/modules/sbpgi-report-and-master-data/sbpgi-report-and-master-data.module.ts	ประกอบ controller/service/providers แล้ว register ที่ app.module.ts
store-backend · src/entities/compensation-documents.entity.ts	entity ของ compensation_documents (@Entity({schema: process.env.DB_SCHEMA}), ไม่ประกาศ relation) — entity รวมหลายเอกสาร: ประกาศครั้งเดียวแล้วอ้างอิง อย่าสร้างซ้ำ
store-backend · src/entities/compensation-histories.entity.ts	entity ของ compensation_histories (@Entity({schema: process.env.DB_SCHEMA}), ไม่ประกาศ relation)
store-backend · src/entities/consideration-logs.entity.ts	entity ของ consideration_logs (@Entity({schema: process.env.DB_SCHEMA}), ไม่ประกาศ relation) — entity รวมหลายเอกสาร: ประกาศครั้งเดียวแล้วอ้างอิง อย่าสร้างซ้ำ
store-backend · src/providers/sbpgi/sbpgi.ts	repository provider แบบ factory ผูก token string กับ DATA_SOURCE — ไฟล์รวมของทุกเอกสาร BE ให้ merge array เพิ่ม ห้ามเขียนทับ
store-backend · sql/deploy-sbpgi-report-and-master-data.sql	DDL production แบบ idempotent (ทีมนี้ไม่ใช่ migration เป็นหลัก)
BFF · src/common/client-services/sbpgi-client.service.ts	client ต่อจาก BaseClientService ตั้ง baseUrl + x-api-key ตอน onModuleInit
BFF · src/modules/sbpgi-report-and-master-data/sbpgi-report-and-master-data.controller.ts	route ผัง BFF prefix /bff/sbpgi/... + @UseGuards(AuthGuard('jwt'))
BFF · src/modules/sbpgi-report-and-master-data/sbpgi-report-and-master-data.service.ts	แนบ x-user-id / x-user-group-id / x-user-permissions แล้ว forward ไป backend

9.2 Controller (store-backend)

```
// src/modules/sbpgi-report-and-master-data/sbpgi-report-and-master-data.controller.ts (ส่วนที่ 1/3 — คลาสเดียวกัน)
import { Body, Controller, Get, Post, Query, UseGuards } from '@nestjs/common';
import { HttpHeaderGuard } from '../../guards/http-header.guard';
import { UserId } from '../../common/decorators/user-id.decorator';
import { SbpgiReportAndMasterDataService } from './sbpgi-report-and-master-data.service';
import { ReportAndMasterDataQueryDto, CreateFactorsBodyDto } from './dto/sbpgi-report-and-master-data.dto';

// LLDD BE - API Report and Master Data
// BFF เรียกด้วย x-api-key และแนบ x-user-id / x-user-group-id / x-user-permissions มาให้
@Controller('sbpgi')
@UseGuards(HttpHeaderGuard)
export class SbpgiReportAndMasterDataController {
  constructor(private readonly service: SbpgiReportAndMasterDataService) {}

  // GET /api/v1/reports/status-summary — รายงานตรวจสอบประกันรายได้
  @Get('reports/status-summary')
  getReportsStatusSummary(@Query() query: ReportAndMasterDataQueryDto, @UserId() userId: string) {
    // TODO: ตรวจสอบ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
    return this.service.getReportsStatusSummary(query, userId);
  }

  // GET /api/v1/reports/status-summary/export — Export Excel
  @Get('reports/status-summary/export')
  exportStatusSummary(@Query() query: ReportAndMasterDataQueryDto, @UserId() userId: string) {
    // TODO: ตรวจสอบ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
    return this.service.exportStatusSummary(query, userId);
  }

  // GET /api/v1/factors — อ่านปัจจัยภายนอก
  @Get('factors')
  getFactors(@Query() query: ReportAndMasterDataQueryDto, @UserId() userId: string) {
    // TODO: ตรวจสอบ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
    return this.service.getFactors(query, userId);
  }

  // POST /api/v1/factors — สร้างปัจจัยภายนอก
  @Post('factors')
  createFactors(@Body() body: CreateFactorsBodyDto, @UserId() userId: string) {
    // TODO: ตรวจสอบ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
    return this.service.createFactors(body, userId);
  }

  // src/modules/sbpgi-report-and-master-data/sbpgi-report-and-master-data.controller.ts (ส่วนที่ 2/3 — คลาสเดียวกัน)
  // import เพิ่ม: UpdateFactorsByCodeBodyDto, CreateCompetitorsBodyDto
  // (method ต่อไปในคลาส SbpgiReportAndMasterDataController เดียวกับส่วนที่ 1)

  // PUT /api/v1/factors/{code} — แก้ปัจจัยภายนอก
  @Put('factors/:code')
  updateFactorsByCode(
```

```

    @Param('code') code: string,
    @Body() body: UpdateFactorsByCodeBodyDto,
    @UserId() userId: string,
  ) {
    // TODO: ตรวจสอบ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
    return this.service.updateFactorsByCode(code, body, userId);
  }

// GET /api/v1/competitors — master แบนด์คู่แข่ง 11 รายการ (รหัส 01-11) — เป็นแหล่งของ dropdown ร...
@Get('competitors')
getCompetitors(@Query() query: ReportAndMasterDataQueryDto, @UserId() userId: string) {
  // TODO: ตรวจสอบ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
  return this.service.getCompetitors(query, userId);
}

// POST /api/v1/competitors — เพิ่มแบนด์คู่แข่ง — code/nameTh/nameEn บังคับ · รหัสซ้ำตอบ 409
@Post('competitors')
createCompetitors(@Body() body: CreateCompetitorsBodyDto, @UserId() userId: string) {
  // TODO: ตรวจสอบ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
  return this.service.createCompetitors(body, userId);
}

// PUT /api/v1/competitors/{code} — แก้ชื่อสถานะ — ห้ามแก้ code เพราะถูกอ้างอิงจาก document_competitors
@Put('competitors/:code')
updateCompetitorsByCode(
  @Param('code') code: string,
  @Body() body: Record<string, unknown>,
  @UserId() userId: string,
) {
  // TODO: ตรวจสอบ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
  return this.service.updateCompetitorsByCode(code, body, userId);
}

// src/modules/sbpgi-report-and-master-data/sbpgi-report-and-master-data.controller.ts (ส่วนที่ 3/3 — คลาสเดียวกัน)
// (method ต่อไปนี้อยู่ในคลาส SbpgiReportAndMasterDataController เดียวกับส่วนที่ 1)

// DELETE /api/v1/competitors/{code} — ลบแบนด์คู่แข่ง — ถูกอ้างในเอกสารแล้วตอบ 409
@Delete('competitors/:code')
removeCompetitorsByCode(@Param('code') code: string, @UserId() userId: string) {
  // TODO: ตรวจสอบ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
  return this.service.removeCompetitorsByCode(code, userId);
}

// DELETE /api/v1/factors/{code} — ลบปัจจัยภายนอกที่ไม่ถูกใช้งาน
@Delete('factors/:code')
removeFactorsByCode(
  @Param('code') code: string,
  @Body() body: Record<string, unknown>,
  @UserId() userId: string,
) {
  // TODO: ตรวจสอบ x-user-permissions ก่อนเรียก service ถ้า endpoint นี้จำกัดสิทธิ์เมนู
  return this.service.removeFactorsByCode(code, body, userId);
}
}

```

9.3 DTO + Validation

```

// src/modules/sbpgi-report-and-master-data/dto/sbpgi-report-and-master-data.dto.ts
import { Type } from 'class-transformer';
import {
  isArray, isBoolean, isIn, isInt, isEmpty, isNumber, isObject, isOptional,
  isString, matches, max, maxLength, min,
} from 'class-validator';

// ValidationPipe ระดับ global ตั้ง whitelist + forbidNonWhitelisted + transform ไว้แล้ว (main.ts)
// property ที่ไม่ประกาศที่นี่จะถูก reject เป็น 400 อัตโนมัติ

// query ร่วมของ GET ทุกเส้นในโมดูลนี้ (path param ใช้ @Param แยก)
export class ReportAndMasterDataQueryDto {
  /** return 400 if missing · BE ผ่าน toAD() เพื่อ client ส่ง พ.ศ. · required เฉพาะบาง endpoint... */
  @IsOptional()
  @Type(() => Number)
  @IsInt()
  year?: number;

  /** 6 สถานะเอกสาร; verbatim จาก sps_store.workflow_status ของ @srm/glb-workflow · required เฉ... */
  @IsOptional()
  @IsString()
  status?: string;

  /** maps to consideration_logs.result_category ล่าสุด · CANCELLED = ยกเลิกโดยระบบ (เพิ่ม 2026... */
  @IsOptional()
  @IsString()

```

```

@IsIn(['APPROVE', 'REJECT', 'CANCELLED', 'PENDING'])
result?: string;

/** 13 region codes; multi-select */
@IsOptional()
@IsArray()
@IsString({ each: true })
region?: string[];

/** **7 คำ** `A B C D E PTT บริษัท` (ยืนยันจาก master `BranchTypeProfile` ของ `CPA_FRN_FGI` ... */
@IsOptional()
@IsArray()
@IsString({ each: true })
storeType?: string[];

/** คง leading zero */
@IsOptional()
@IsString()
@Matches(/^\d{5}$/, { message: 'รหัสร้านต้องเป็นตัวเลข 5 หลัก และคงเลขศูนย์นำหน้า' })
impactedStoreCode?: string;

// TODO: เพิ่ม property ที่เหลือของ payload นี้ให้ครบตามหัวข้อฟิลด์ของเอกสารนี้
,

// body ของ POST /api/v1/factors
export class CreateFactorsBodyDto {
  @IsNotEmpty()
  @IsString()
  factorCode: string;

  @IsNotEmpty()
  @IsString()
  factorName: string;

  @IsNotEmpty()
  @IsString()
  description: string;

  @IsNotEmpty()
  @Type(() => Boolean)
  @IsBoolean()
  active: boolean;

  /** audit reason */
  @IsNotEmpty()
  @IsString()
  @MaxLength(500)
  reason: string;
,

// body ของ PUT /api/v1/factors/{code}
export class UpdateFactorsByCodeBodyDto {
  @IsNotEmpty()
  @IsString()
  factorName: string;

  @IsNotEmpty()
  @IsString()
  description: string;

  @IsNotEmpty()
  @Type(() => Boolean)
  @IsBoolean()
  active: boolean;

  /** audit reason */
  @IsNotEmpty()
  @IsString()
  @MaxLength(500)
  reason: string;
,

// body ของ POST /api/v1/competitors
export class CreateCompetitorsBodyDto {
  @IsNotEmpty()
  @IsString()
  code: string;

  @IsNotEmpty()
  @IsString()
  nameTh: string;

  @IsNotEmpty()
  @IsString()
  nameEn: string;

```

```

@IsNotEmpty()
@IsString()
remark: string;
}

```

// TODO: สร้าง BodyDto ของ endpoint ที่เหลือด้วยรูปแบบเดียวกัน: PUT /api/v1/competitors/{code}, DELETE /api/v1/factors/{code}

9.4 Service (inject `DATA\_SOURCE` + raw SQL)

service ประกาศ method ครบทุกเส้นที่ controller เรียก และ signature มาจากแหล่งเดียวกับ controller (จำนวน/ลำดับพารามิเตอร์จึงตรงกันเสมอ) — เส้นที่ยังไม่ได้ implement เป็น stub ที่ throw new NotImplementedException(. . .) ให้ TypeScript compile ผ่านตั้งแต่วันแรก

```

// src/modules/sbpgi-report-and-master-data/sbpgi-report-and-master-data.service.ts
import { Inject, Injectable, Logger, NotFoundException, NotImplementedException } from '@nestjs/common';
import { DataSource } from 'typeorm';
import { SBPGI_SQL } from './sbpgi-report-and-master-data.sql';

@Injectable()
export class SbpqiReportAndMasterDataService {
  private readonly logger = new Logger(SbpqiReportAndMasterDataService.name);

  constructor(
    // DATA_SOURCE override query(): SELECT/WITH ไป slave pool, write ไป master
    @Inject('DATA_SOURCE') private readonly dataSource: DataSource,
  ) {}

  // GET /api/v1/reports/status-summary — รายงานตรวจสอบประกันรายได้
  async getReportsStatusSummary(query: ReportAndMasterDataQueryDto, userId: string) {
    const page = Number(query.page ?? 1);
    const size = Math.min(Number(query.size ?? 20), 100);
    // SQL เต็มอยู่ในหัวข้อ Database SQL ของเอกสารนี้ (คือ 'GET /api/v1/reports/status-summary')
    // □□ SQL ตัวอย่างบางเส้นเขียนด้วย named parameter (:size/:offset) แต่ dataSource.query()
    // รับเฉพาะ positional $1..$n — ต้องแปลงชื่อเป็นลำดับก่อน หรือใช้ QueryBuilder แทน
    const rows = await this.dataSource.query(SBPGI_SQL.getReportsStatusSummary, [
      // TODO: เรียงพารามิเตอร์ให้ตรงกับ $1..$n ของ SQL จริง
      userId, (page - 1) * size, size,
    ]);
    // TODO: total ต้องมาจาก COUNT(*) แยก query หรือ window function ไม่ใช่ rows.length
    return { page, size, total: rows.length, items: rows };
  }

  // GET /api/v1/reports/status-summary/export — Export Excel
  async exportStatusSummary(query: ReportAndMasterDataQueryDto, userId: string) {
    // TODO: implement ตาม business rule ของ GET /api/v1/reports/status-summary/export
    // (SQL อยู่ในหัวข้อ Database SQL คือ 'GET /api/v1/reports/status-summary/export')
    throw new NotImplementedException('exportStatusSummary ยังไม่ implement');
  }

  // GET /api/v1/factors — อ่านปัจจัยภายนอก
  async getFactors(query: ReportAndMasterDataQueryDto, userId: string) {
    // TODO: implement ตาม business rule ของ GET /api/v1/factors
    // (SQL อยู่ในหัวข้อ Database SQL คือ 'GET /api/v1/factors')
    throw new NotImplementedException('getFactors ยังไม่ implement');
  }

  // POST /api/v1/factors — สร้างปัจจัยภายนอก
  // mutation ต้องอยู่ใน transaction เดียว (ไม่มี audit ของ master แล้ว · 2026-08-07)
  async createFactors(body: CreateFactorsBodyDto, userId: string) {
    const runner = this.dataSource.createQueryRunner();
    await runner.connect();
    await runner.startTransaction();
    try {
      // TODO: lock แถวเป้าหมายของ external_factors ด้วย SELECT ... FOR UPDATE ก่อนเขียน
      const [current] = await runner.query(SBPGI_SQL.createFactorsLock, [body.docNo]);
      if (!current) {
        throw new NotFoundException('ไม่พบข้อมูลที่ต้องการ');
      }
      await runner.query(SBPGI_SQL.createFactors, [/* TODO: ผู้คํ้าจาก body */]);
      await runner.commitTransaction();
      return { message: 'saved' };
    } catch (error) {
      await runner.rollbackTransaction();
      this.logger.error(error);
      throw error;
    } finally {
      await runner.release();
    }
  }
}

```



```

// PUT /api/v1/factors/{code} — แก้ปัจจัยภายนอก
async updateFactorsByCode(code: string, body: UpdateFactorsByCodeBodyDto, userId: string) {
  // TODO: implement ตาม business rule ของ PUT /api/v1/factors/{code}
  // (SQL อยู่ในหัวข้อ Database SQL คีย์ 'PUT /api/v1/factors/{code}')
  throw new NotImplementedException('updateFactorsByCode ยังไม่ implement');
}

// GET /api/v1/competitors — master แบนด์คู่แข่ง 11 รายการ (รหัส 01-11) — เป็นแหล่งของ dropdown ร...
async getCompetitors(query: ReportAndMasterDataQueryDto, userId: string) {
  // TODO: implement ตาม business rule ของ GET /api/v1/competitors
  // (SQL อยู่ในหัวข้อ Database SQL คีย์ 'GET /api/v1/competitors')
  throw new NotImplementedException('getCompetitors ยังไม่ implement');
}

// POST /api/v1/competitors — เพิ่มแบนด์คู่แข่ง — code/nameTh/nameEn บังคับ · รหัสซ้ำตอบ 409
async createCompetitors(body: CreateCompetitorsBodyDto, userId: string) {
  // TODO: implement ตาม business rule ของ POST /api/v1/competitors
  // (SQL อยู่ในหัวข้อ Database SQL คีย์ 'POST /api/v1/competitors')
  throw new NotImplementedException('createCompetitors ยังไม่ implement');
}

// PUT /api/v1/competitors/{code} — แก้ชื่อ/สถานะ — ห้ามแก้ code เพราะถูกอ้างอิงจาก document_competitors
async updateCompetitorsByCode(code: string, body: Record<string, unknown>, userId: string) {
  // TODO: implement ตาม business rule ของ PUT /api/v1/competitors/{code}
  // (SQL อยู่ในหัวข้อ Database SQL คีย์ 'PUT /api/v1/competitors/{code}')
  throw new NotImplementedException('updateCompetitorsByCode ยังไม่ implement');
}

// DELETE /api/v1/competitors/{code} — ลบแบนด์คู่แข่ง — ถูกอ้างอิงเอกสารแล้วตอบ 409
async removeCompetitorsByCode(code: string, userId: string) {
  // TODO: implement ตาม business rule ของ DELETE /api/v1/competitors/{code}
  // (SQL อยู่ในหัวข้อ Database SQL คีย์ 'DELETE /api/v1/competitors/{code}')
  throw new NotImplementedException('removeCompetitorsByCode ยังไม่ implement');
}

// DELETE /api/v1/factors/{code} — ลบปัจจัยภายนอกที่ไม่ถูกใช้งาน
async removeFactorsByCode(code: string, body: Record<string, unknown>, userId: string) {
  // TODO: implement ตาม business rule ของ DELETE /api/v1/factors/{code}
  // (SQL อยู่ในหัวข้อ Database SQL คีย์ 'DELETE /api/v1/factors/{code}')
  throw new NotImplementedException('removeFactorsByCode ยังไม่ implement');
}
}

```

9.5 Entity (TypeORM)

```

// src/entities/compensation-documents.entity.ts
import { Column, Entity, PrimaryColumn } from 'typeorm';

@Entity({ name: 'compensation_documents', schema: process.env.DB_SCHEMA })
export class CompensationDocument {
  @PrimaryColumn({ name: 'doc_no', type: 'varchar', length: 12 })
  docNo: string;

  @Column({ name: 'impact_process_id', type: 'bigint', nullable: true })
  impactProcessId?: number;

  @Column({ name: 'impacted_store_code', type: 'char', length: 5 })
  impactedStoreCode: string;

  @Column({ name: 'status_code', type: 'varchar', length: 2 })
  statusCode: string;

  @Column({ name: 'current_section_code', type: 'varchar', length: 2 })
  currentSectionCode: string;

  @Column({ name: 'round_no', type: 'int', nullable: true })
  roundNo?: number;

  @Column({ name: 'loop_no', type: 'int', nullable: true })
  loopNo?: number;

  @Column({ name: 'statement_id', type: 'varchar', length: 30, nullable: true })
  statementId?: string;

  @Column({ name: 'statement_date', type: 'date', nullable: true })
  statementDate?: Date;

  @Column({ name: 'account_year', type: 'int', nullable: true })
  accountYear?: number;

  @Column({ name: 'account_month', type: 'int', nullable: true })
  accountMonth?: number;
}

```

```

@Column({ name: 'compensate_amount', type: 'numeric', precision: 15, scale: 2, nullable: true })
compensateAmount?: string;

@Column({ name: 'allmap_url', type: 'text', nullable: true })
allmapUrl?: string;

@Column({ name: 'approver_snapshot', type: 'jsonb', nullable: true })
approverSnapshot?: Record<string, unknown>;

@Column({ name: 'created_at', type: 'timestampz', nullable: true })
createdAt?: Date;

@Column({ name: 'updated_at', type: 'timestampz', nullable: true })
updatedAt?: Date;

// TODO: ตรวจสอบความยาว/precision กับ DDL จริงใน sql/deploy-sbpgi-*.sql ก่อน merge
// entity ชุดนี้ไม่ประกาศ relation ตาม convention (join ด้วย raw SQL)
,
// src/entities/compensation-histories.entity.ts
import { Column, Entity, PrimaryColumn } from 'typeorm';

@Entity({ name: 'compensation_histories', schema: process.env.DB_SCHEMA })
export class CompensationHistory {
  @PrimaryColumn({ name: 'id', type: 'bigint' })
  id: number;

  @Column({ name: 'store_code', type: 'char', length: 5 })
  storeCode: string;

  @Column({ name: 'ref_doc_no', type: 'varchar', length: 12, nullable: true })
  refDocNo?: string;

  @Column({ name: 'compensate_year', type: 'int' })
  compensateYear: number;

  @Column({ name: 'compensate_month', type: 'int' })
  compensateMonth: number;

  @Column({ name: 'compensate_amount', type: 'numeric', precision: 15, scale: 2 })
  compensateAmount: string;

  @Column({ name: 'submit_account_month', type: 'varchar', length: 7, nullable: true })
  submitAccountMonth?: string;

  @Column({ name: 'submit_status', type: 'char', length: 1, nullable: true })
  submitStatus?: string;

  // TODO: ตรวจสอบความยาว/precision กับ DDL จริงใน sql/deploy-sbpgi-*.sql ก่อน merge
  // entity ชุดนี้ไม่ประกาศ relation ตาม convention (join ด้วย raw SQL)
}

```

ตารางที่เหลือนี้ของเอกสารนี้ (consideration_logs, glb-workflow, external_factors, competitors, document_competitors) ใช้รูปแบบ entity เดียวกัน — คอลัมน์อ้างอิงจาก Database design

ตารางที่ไม่ต้องสร้าง entity เพราะใช้ของระบบเดิม/workflow engine:

Object	R/W	ใช้ของระบบเดิมตัวไหน
mas_param	R	mas_param (store-backend)

9.6 Repository Providers + Module wiring

```

// src/providers/sbpgi/sbpgi.ts — repository provider แบบ factory (ไม่ใช่ TypeOrmModule.forFeature)
// convention ของโฟลเดอร์ providers คือ 1 โฟลเดอร์ต่อโดเมน ตั้งชื่อตามโดเมน (business_user/business_user.ts,
// common_code/common_code.ts ...) ไม่ใช่ index.ts
//
// □□ โฟลเดอร์นี้ใช้ร่วมกันทุกเอกสาร BE ของ SBPGI — ให้ **merge array เพิ่ม** เข้าไฟล์เดิม ห้ามเขียนทับ
// (ชื่อ const แยกต่อเอกสารไว้แล้วเพื่อไม่ให้ชนกัน)
import { DataSource } from 'typeorm';
import { CompensationDocument } from '../entities/compensation-documents.entity';
import { CompensationHistory } from '../entities/compensation-histories.entity';
import { ConsiderationLog } from '../entities/consideration-logs.entity';

export const sbpgiReportAndMasterDataProviders = [
  {
    provide: 'COMPENSATION_DOCUMENT_REPOSITORY',
    useFactory: (dataSource: DataSource) => dataSource.getRepository(CompensationDocument),
    inject: ['DATA_SOURCE'],
  },
],

```

```

{
  provide: 'COMPENSATION_HISTORIES_REPOSITORY',
  useFactory: (dataSource: DataSource) => dataSource.getRepository(CompensationHistory),
  inject: ['DATA_SOURCE'],
},
{
  provide: 'CONSIDERATION_LOG_REPOSITORY',
  useFactory: (dataSource: DataSource) => dataSource.getRepository(ConsiderationLog),
  inject: ['DATA_SOURCE'],
},
];

// src/modules/sbpgi-report-and-master-data/sbpgi-report-and-master-data.module.ts
import { MiddlewareConsumer, Module, NestModule } from '@nestjs/common';
import { DatabaseModule } from '../database/database.module';
// UserContextMiddleware อ่าน header x-user-id แล้วเซต request.userId ที่ @UserId() ใช้
// — app.module.ts **ไม่ได้** apply แบบ global (มีแค่ HttpContext/LoggerContext) แต่ละโมดูลต้อง apply เอง
// (ดู evaluation-process.module.ts / inform-evaluate.module.ts / cooperation-request.module.ts)
import { UserContextMiddleware } from '../common/middleware/user-context.middleware';
import { SbpgiReportAndMasterDataProviders } from '../providers/sbpgi/sbpgi';
import { SbpgiReportAndMasterDataController } from './sbpgi-report-and-master-data.controller';
import { SbpgiReportAndMasterDataService } from './sbpgi-report-and-master-data.service';

@Module({
  imports: [DatabaseModule],
  controllers: [SbpgiReportAndMasterDataController],
  providers: [SbpgiReportAndMasterDataService, ...sbpgiReportAndMasterDataProviders],
  exports: [SbpgiReportAndMasterDataService],
})
export class SbpgiReportAndMasterDataModule implements NestModule {
  configure(consumer: MiddlewareConsumer) {
    // ถ้าไม่ apply ตรงนี้ userId จะเป็น undefined เจียน ๆ ทุก endpoint
    consumer.apply(UserContextMiddleware).forRoutes(SbpgiReportAndMasterDataController);
  }
}
// TODO: register module นี้ใน app.module.ts (imports) พร้อมกับโมดูล SBPGI ตัวอื่น

```

9.7 BFF Proxy (module + controller + client service)

BFF ยังไม่มีที่เจอรูปประกันรายได้เลย จึงต้องสร้าง module ใหม่ + client service ใหม่ทั้งชุด และเลือก prefix แบบเดียวทั้งโมดูล (ที่นี้ใช้ `/bff/sbpgi/...`) เพื่อไม่ให้ปนแบบที่มี/ไม่มี `/bff` เหมือนโมดูลเดิม

```

// src/common/client-services/sbpgi-client.service.ts
import { Injectable, Logger, OnModuleInit } from '@nestjs/common';
import { BaseClientService } from './base-client.service';

@Injectable()
export class SbpgiClientService extends BaseClientService implements OnModuleInit {
  protected logger: Logger = new Logger(SbpgiClientService.name);

  onModuleInit() {
    // TODO: ถ้า deploy SBPGI แยก service ให้เพิ่ม API_SBPGI_BACKEND_* ใน AppConfigService
    // ตอนนี้ชี้ store backend ตัวเดียวกับ StoreClientService
    this.defaultHeaders[this.config.api.store.key.name] = this.config.api.store.key.value;
    this.baseUrl = this.config.api.store.url;
  }
}

// BaseClientService และ { success, data } ให้แล้ว — service ฟัง BFF จึงได้ data ตรง ๆ
// TODO: เขียน SbpgiClientService ใน providers/exports และ ClientServiceModule (Global)

// src/modules/sbpgi-report-and-master-data/sbpgi-report-and-master-data.service.ts (BFF)
import { Injectable } from '@nestjs/common';
import { SbpgiClientService } from '@common/client-services/sbpgi-client.service';

@Injectable()
export class SbpgiReportAndMasterDataBffService {
  constructor(private readonly client: SbpgiClientService) {}

  // BFF ไม่มี DB — หน้าทีเดียวคือแนบ user context แล้ว forward
  private userHeaders(user: any) {
    return {
      'x-user-id': user?.userId,
      'x-user-group-id': user?.groupId,
      'x-user-permissions': (user?.permissions ?? []).join(','),
    };
  }

  getReportsStatusSummary(params: any, user: any) {
    return this.client.get('/api/v1/reports/status-summary', { params, headers: this.userHeaders(user) });
  }

  exportStatusSummary(params: any, user: any) {
    return this.client.get('/api/v1/reports/status-summary/export', { params, headers: this.userHeaders(user) });
  }
}

```

```

}

getFactors(params: any, user: any) {
  return this.client.get('/api/v1/factors', { params, headers: this.userHeaders(user) });
}
}

// ----- src/modules/sbpgi-report-and-master-data/sbpgi-report-and-master-data.controller.ts (BFF) -----
import { Body, Controller, Delete, Get, Param, Post, Put, Query, Req, UseGuards } from '@nestjs/common';
import { AuthGuard } from '@nestjs/passport';

// เลือก prefix แบบเดียวทั้งโมดูล: ใช้ '/bff/sbpgi/...' (ห้ามปนกับแบบไม่มี /bff)
@Controller('bff/sbpgi/report-and-master-data')
@UseGuards(AuthGuard('jwt'))
export class SbpgiReportAndMasterDataBffController {
  constructor(private readonly service: SbpgiReportAndMasterDataBffService) {}

  // proxy ของ GET /api/v1/reports/status-summary
  @Get('reports/status-summary')
  getReportsStatusSummary(@Query() query: any, @Req() req: any) {
    return this.service.getReportsStatusSummary(query, req.user);
  }

  // proxy ของ GET /api/v1/reports/status-summary/export
  @Get('reports/status-summary/export')
  exportStatusSummary(@Query() query: any, @Req() req: any) {
    return this.service.exportStatusSummary(query, req.user);
  }
}

// TODO: register module ใน app.module.ts ของ BFF และเพิ่ม SbpgiClientService ใน ClientServiceModule (@Global)

```

10. Database SQL

10.1 ตารางที่อ่าน/เขียน

Table / Object	R/W	Usage
compensation_documents	R	แหล่งข้อมูลรายงานและ filter status/year
compensation_histories	R	ยอดเงินชดเชยและงวด statement
consideration_logs	R	ผลพิจารณาล่าสุด APPROVE/REJECT
glb-workflow	R	ผู้ปฏิบัติงาน — ตาราง operator_assignments ถูกตัด 2026-08-05
external_factors	R/W	master บัญชีภายนอก
competitors	R/W	master แปรนัยคู่แข่ง 11 รายการ (code 01-11 · name_th · name_en · remark) — feed dropdown ร้านคู่แข่งของหน้าเอกสาร
document_competitors	R	ตรวจว่าแบรนด์ถูกอ้างในเอกสารก่อนลบ (409)
mas_param	R	ใช้ของระบบเดิม: mas_param (store-backend)

10.2 SQL จริงต่อ Endpoint

GET /api/v1/reports/status-summary — รายงานตรวจสอบประกันรายได้

```

-- □□ ชื่อคอลัมน์ต่อไปนี้ไม่ตรงกับ entity ที่หัวข้อ Entity ของเอกสารนี้ประกาศไว้:
-- d.year -> d.account_year
-- □□ SQL นี้ใช้ named parameter (:name) แต่ `dataSource.query()` ของ store-backend
-- รับเฉพาะ positional $1..$n — ต้องแปลงเป็นลำดับ หรือรันผ่าน QueryBuilder
-- 14 คอลัมน์ตาม SDD สไลด์ 60 ; ต่อระบุ :year และ :status เสมอ ; เอาเฉพาะเอกสารที่มีเลขที่แล้ว
-- □□ ตาราง stores ของ SBPGI ถูกตัด 2026-08-06 — ใช้ store ของระบบ SBP เดิม (sps_store 19,402 แถว): คีย์ store_id · ภาค zone_cd
SELECT si.store_id AS impacted_store_code, si.store_name AS impacted_store_name,
       si.zone_cd AS impacted_region, si.store_type AS impacted_store_type,
       pr.impact_month, pr.impact_year, d.statement_date, -- statement_date = ค.ศ. (คอลัมน์ใหม่คู่กับ statement_id)
       ns.store_id AS new_store_code, ns.store_name AS new_store_name,
       ns.zone_cd AS new_region, ns.store_type AS new_store_type,
       h.compensate_amount, d.loop_no AS round_no, d.created_at AS created_date, d.doc_no
FROM compensation_documents d
JOIN fgi_impact_processes pr ON pr.id = d.impact_process_id
JOIN store si ON si.store_id = d.impacted_store_code
LEFT JOIN compensation_histories h ON h.ref_doc_no = d.doc_no
LEFT JOIN document_new_stores dns ON dns.doc_no = d.doc_no
LEFT JOIN store ns ON ns.store_id = dns.new_store_code

```

```

LEFT JOIN LATERAL (
  SELECT result_category FROM consideration_logs
  WHERE doc_no = d.doc_no ORDER BY action_datetime DESC LIMIT 1
) cl ON TRUE
WHERE d.year = :year
AND d.status_code = :status -- Drop-down บังคับ (SDD สไลด์ 60)
AND (:impactedStoreCode IS NULL OR d.impacted_store_code = :impactedStoreCode)
AND (:newStoreCode IS NULL OR dns.new_store_code = :newStoreCode)
AND (:psFrom IS NULL OR d.statement_date BETWEEN :psFrom AND :psTo) -- ค.ศ. ; บังคับเมื่อ status = เสร็จสิ้นดำเนินการ
AND (:storeTypes IS NULL OR si.store_type = ANY(:storeTypes)) -- 7 ค่า 'A B C D E PTT บริษัท' (BranchTypeProfile.BranchTypeFGName ·
ห้าม hardcode)
AND (:regions IS NULL OR si.zone_cd = ANY(:regions)) -- 13 ภาค + ภาคใหม่อัตโนมัติ
AND (:result IS NULL OR cl.result_category = :result) -- APPROVE / REJECT (ไม่บังคับ)
ORDER BY d.doc_no
LIMIT :size OFFSET :offset;

```

GET /api/v1/reports/status-summary/export — Export Excel

```

-- เงื่อนไขเดียวกับ status-summary ทุกตัว แล้ว stream 14 คอลัมน์เดิมออกเป็นไฟล์ .xlsx (Export Excel)
-- ใช้ SELECT ชุดเดียวกับ GET /reports/status-summary แต่ไม่ตัดหน้า (ไม่มี LIMIT/OFFSET)
ORDER BY d.doc_no;

```

GET /api/v1/factors — อ่านปัจจัยภายนอก

```

-- □□ SQL นี้ใช้ named parameter (:name) แต่ `dataSource.query()` ของ store-backend
-- รับเฉพาะ positional $1..$n — ต้องแปลงเป็นลำดับ หรือรันผ่าน QueryBuilder
SELECT factor_code, factor_name, factor_remark
FROM external_factors
WHERE :q IS NULL OR factor_name LIKE :q
ORDER BY factor_code;

```

POST /api/v1/factors — สร้างปัจจัยภายนอก

```

-- □□ SQL นี้ใช้ named parameter (:name) แต่ `dataSource.query()` ของ store-backend
-- รับเฉพาะ positional $1..$n — ต้องแปลงเป็นลำดับ หรือรันผ่าน QueryBuilder
-- factor_code ห้ามซ้ำ (ไม่จัน 409)
INSERT INTO external_factors (factor_code, factor_name, factor_remark)
VALUES (:factorCode, :factorName, :factorRemark);

```

PUT /api/v1/factors/{code} — แก้ปัจจัยภายนอก

```

-- □□ SQL นี้ใช้ named parameter (:name) แต่ `dataSource.query()` ของ store-backend
-- รับเฉพาะ positional $1..$n — ต้องแปลงเป็นลำดับ หรือรันผ่าน QueryBuilder
-- ไม่มี audit/เหตุผลแล้ว (ยกเลิก audit_logs 2026-08-07)
UPDATE external_factors SET factor_name = :factorName, factor_remark = :factorRemark
WHERE factor_code = :code;

```

GET /api/v1/competitors — master แบรินด์คู่แข่ง 11 รายการ (รหัส 01-11) — เป็นแหล่งของ dropdown ร้านคู่แข่งในหน้าเอกสารด้วย

```

-- □□ SQL นี้ใช้ named parameter (:name) แต่ `dataSource.query()` ของ store-backend
-- รับเฉพาะ positional $1..$n — ต้องแปลงเป็นลำดับ หรือรันผ่าน QueryBuilder
-- master แบรินด์คู่แข่ง 11 รายการ (รหัส 01-11) · ระบบเดิมเก็บชื่อไทยและอังกฤษ
SELECT competitor_code, name_th, name_en, remark, is_active
FROM competitors
WHERE (:q IS NULL OR name_th LIKE :q OR name_en LIKE :q)
ORDER BY competitor_code;

```

POST /api/v1/competitors — เพิ่มแบรินด์คู่แข่ง — code/nameTh/nameEn บังคับ · รหัสซ้ำตอบ 409

```

-- □□ SQL นี้ใช้ named parameter (:name) แต่ `dataSource.query()` ของ store-backend
-- รับเฉพาะ positional $1..$n — ต้องแปลงเป็นลำดับ หรือรันผ่าน QueryBuilder
-- competitor_code ห้ามซ้ำ (ไม่จัน 409) · ชื่อไทยและอังกฤษบังคับทั้งคู่
INSERT INTO competitors (competitor_code, name_th, name_en, remark, is_active)
VALUES (:code, :nameTh, :nameEn, :remark, TRUE);

```

PUT /api/v1/competitors/{code} — แก้ชื่อ/สถานะ — ห้ามแก้ code เพราะถูกอ้างอิงจาก document_competitors

```

-- □□ SQL นี้ใช้ named parameter (:name) แต่ `dataSource.query()` ของ store-backend
-- รับเฉพาะ positional $1..$n — ต้องแปลงเป็นลำดับ หรือรันผ่าน QueryBuilder
-- ห้ามแก้ competitor_code (เป็น PK และถูกอ้างอิงจาก document_competitors)
UPDATE competitors
SET name_th = :nameTh, name_en = :nameEn, remark = :remark, is_active = :isActive,
    updated_at = CURRENT_TIMESTAMP
WHERE competitor_code = :code;

```

DELETE /api/v1/competitors/{code} — ลบแบรินด์คู่แข่ง — ถูกอ้างอิงในเอกสารแล้วตอบ 409

```

-- □□ SQL นี้ใช้ named parameter (:name) แต่ `dataSource.query()` ของ store-backend
-- รับเฉพาะ positional $1..$n — ต้องแปลงเป็นลำดับ หรือรันผ่าน QueryBuilder
-- ตรวจสอบถูกอ้างอิงในเอกสารก่อนลบ (ไม่จัน 409)
SELECT 1 FROM document_competitors WHERE competitor_code = :code;

DELETE FROM competitors WHERE competitor_code = :code;

```

DELETE /api/v1/factors/{code} — ลบปัจจัยภายนอกที่ไม่ถูกใช้งาน

```
-- □□ SQL นี้ใช้ named parameter (:name) แต่ `dataSource.query()` ของ store-backend
-- รับเฉพาะ positional $1..$n — ต้องแปลงเป็นลำดับ หรือรันผ่าน QueryBuilder
-- ตรวจสอบไม่ถูกอ้างในเอกสารก่อนลบ (ไม่ขึ้น 409)
SELECT 1 FROM document_external_factors WHERE factor_code = :code;
```

```
DELETE FROM external_factors WHERE factor_code = :code;
```

10.3 Index / Constraint ที่ควรมี (ข้อเสนอ)

Table	DDL ที่เสนอ	ที่มา / หมายเหตุ
compensation_histories	CREATE INDEX idx_compensation_histories_ref_doc_no ON compensation_histories (ref_doc_no);	ข้อเสนอ — อนุมานจากคอลัมน์ที่ปรากฏใน WHERE/JOIN ของ SQL ด้านบน ต้องวัด EXPLAIN ก่อนใช้จริง
compensation_documents	CREATE INDEX idx_compensation_docume nts_year_status_code_impacted_store_code ON compensation_documents (year, status_code, impacted_store_code);	ข้อเสนอ — อนุมานจากคอลัมน์ที่ปรากฏใน WHERE/JOIN ของ SQL ด้านบน ต้องวัด EXPLAIN ก่อนใช้จริง

ทั้งหมดเป็น ข้อเสนอ ไม่ใช่ข้อกำหนดจาก ข้อกำหนดทางธุรกิจ — ให้ตรวจกับ EXPLAIN ANALYZE บนข้อมูลจริง และรวมเข้าไฟล์
sql/deploy-sbpgi-*.sql แบบ idempotent (CREATE INDEX IF NOT EXISTS) ตาม pattern ที่ทีมใช้อยู่

11. Processing Flow

Step	Description
1	Validate filter
2	Build query
3	Apply pagination/export mode
4	Return rows or CSV
5	For mutations validate reason and write audit

12. Acceptance Criteria

- missing year/status/result fails
- export uses same filters as preview
- master edit requires reason
- config locked value cannot edit

13. Developer Test Checklist

No	Test
1	report missing year
2	report export
3	factor duplicate
4	operator audit
5	config locked

14. Unit Test Scope

9 ชั่วโมง (30% ของ implementation 30 ชั่วโมง) · เครื่องมือ: Jest + mock repository/DataSource (ไม่ต่อ DB จริง)

หัวข้อนี้คือ unit test ที่ต้องเขียนคู่กับโค้ด — ต่างจาก *Developer Test Checklist* ซึ่งเป็น scenario ระดับ end-to-end/manual ที่ใช้ตอนตรวจรับ · รายการด้านล่าง derive จาก field/validation, acceptance criteria, endpoint และตารางที่เอกสารนี้เขียน

สิ่งที่ทดสอบ	ประเภท	เกณฑ์ผ่าน
year	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: required for report · รูปแบบ: ค.ศ. YYYY
status	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: required · รูปแบบ: statusCode string
result	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: optional for report (บังคับเฉพาะ status) · รูปแบบ: APPROVE REJECT CANCELLED PENDING
reason	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: required mutation · รูปแบบ: text
page/size	validation	ผ่านเมื่อถูกกฎ / โยน error เมื่อผิด — กฎ: page>=1 size<=100 · รูปแบบ: integer
business rule	logic	missing year/status/result fails
business rule	logic	export uses same filters as preview
business rule	logic	master edit requires reason
business rule	logic	config locked value cannot edit
GET /api/v1/reports/status-summary	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง
GET /api/v1/reports/status-summary/export	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง
GET /api/v1/factors	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง
POST /api/v1/factors	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง
PUT /api/v1/factors/{code}	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง
GET /api/v1/competitors	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง
POST /api/v1/competitors	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง
PUT /api/v1/competitors/{code}	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง

สิ่งที่ทดสอบ	ประเภท	เกณฑ์ผ่าน
DELETE /api/v1/competitors/{code}	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง
DELETE /api/v1/factors/{code}	handler	คืน {success:true,data} ตามรูปแบบที่ระบุ และคืน {success:false,error:{code,message}} เมื่อ input ผิด — mock repository/lib ไม่แตะ DB จริง
external_factors, competitors	transaction	จำลอง error กลางทาง แล้วยืนยันว่า rollback ครบ ไม่เหลือแถวค้าง (mock DataSource/QueryRunner)
service	error mapping	แปลง error ของ repository/lib เป็น error code ตามสัญญากลาง (LLDD-BE-API-Common-Contracts.pdf)

- ทุกเคสต้องรันได้โดยไม่ต่อ DB/บริการภายนอกจริง — mock ที่ขอบ repository/client เสมอ
- ข้อความไทยที่ยืนยันในเทสต้องเป็น verbatim ตาม ข้อกำหนดทางธุรกิจ ห้ามพิมพ์ใหม่
- เกณฑ์ผ่านของ CI: ทุกเคสในตารางนี้มี test จริงและผ่านทั้งหมด